Chapitre 6

ANALYSES LEXICALE, SYNTAXIQUE ET SEMANTIQUE

Partie I - Analyse lexicale

1. Introduction

Les éléments rencontrés dans un programme :

- Identificateur : nom du programme, X, Y, …
- Mots clés : si, sinon, tant que, fin, …
- Constantes : 12, 375, …
- Séparateur : +, -, ., >, <, …
- Commentaires : { }, * *, /* */, …

Le rôle de l'analyse lexicale (le « lexer» ou « scanner») est de reconnaître les caractères du programme, de les regrouper pour former des unités lexicales et d'éliminer les caractères inutiles tels que les commentaires, les espaces, les tabulations … Elle transforme le texte en une suite de lexèmes (token en anglais). Le programme est donc découpé en morceaux (identificateurs, entiers littéraux, ponctuation)

Flot d'entrée est le texte source, caractère par caractère.

Flot de sortie est l'ensemble des unités lexicales ou lexèmes (« tokens »).

Unité lexicale : désigne l'ensemble des lexèmes ayant la même signification grammaticale.

Exemple

Soit l'instruction suivante :
if (x1 * x2 < 1.0) {
y = x1 ;}

Le flot d'entrée est le suivant :



Après l'analyse lexicale, le flot de sortie est :

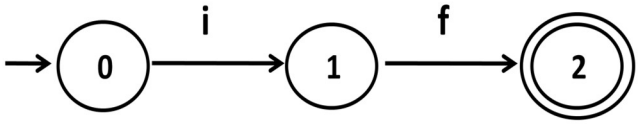
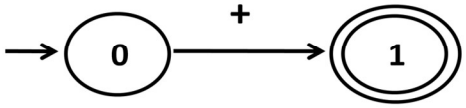
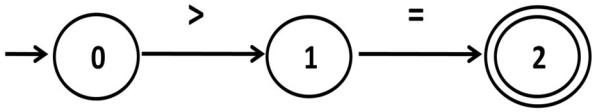


2. Structure d'un analyseur lexical

L'analyseur lexical est basé sur les expressions régulières. Il est donc un automate fini déterministe qui reconnaît l'union des langages des lexèmes avec un état terminal correspondant à la reconnaissance d'un mot d'un des langages des lexèmes.

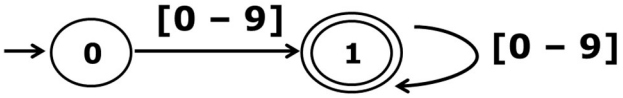
Il utilise un programme squelette commun à tous les analyseurs lexicaux et une table dans laquelle est stockée la fonction de transition de l'automate associé.

2. Reconnaissance des lexèmes

Lexème	Expression régulière	automate
Mot clé **if**	**if**	
Symbole +	+	
Symbole >=	>=	

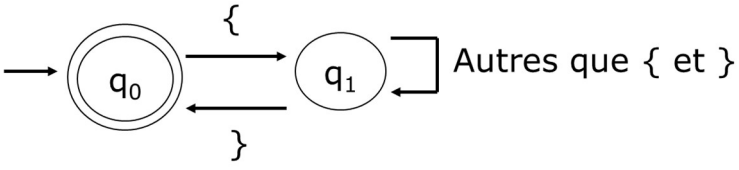
Constantes entières

Ces constantes sont représentées par l'expression régulière suivante : [0 - 9][0 – 9]*



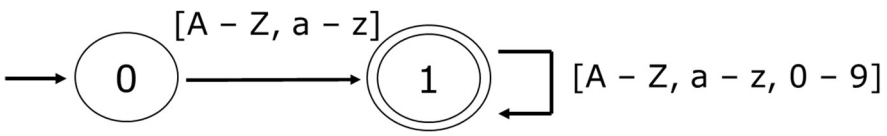
Commentaires

Un automate qui reconnait les commentaires.



Identificateurs

Les identificateurs sont représentés par l'expression régulière suivante : [A – Z, a - z][A – Z , a – z , 0 - 9]*



4. Reconnaissance contextuelle

Il existe des langages où une forme lexicale peut correspondre à différents lexèmes selon le contexte.

Remarque

Nous avons donc intérêt à ne faire reconnaître par l'automate que les identificateurs, puis à regarder dans une table si l'identificateur reconnu correspond à un mot clé. La recherche dans cette table peut être rapide (par dichotomie, par adressage dispersé . . .)

5. Génération d'analyseurs

Un analyseur lexical est un automate fini pour la réunion de toutes les expressions régulières définissant les lexèmes. Pour produire un analyseur à partir d'une spécification, il faut passer par les étapes suivantes :

- Traduction d'expressions rationnelles en automates finis non-déterministes ;
- Déterminisation ;
- Minimisation ;
- Génération directe de l'automate fini déterministe minimal.

Programme 1

Char ← prochain caractère
Etat ← q_0
Tant que (char ≠ EOF)
Etat ← δ (Etat, char)
Char ← prochain caractère
Si (Etat ∈ Q_F)
Alors accepter
Sinon rejeter

Remarque

Ces analyseurs passent beaucoup de temps à tester et à manipuler la variable d'état. Ce temps peut être optimisé en utilisant les analyseurs lexicaux codés directement.

6. Analyseur lexical codé directement

Soit l'automate A cité ci-dessus, l'analyseur lexical sera codé comme :

q_0 : Char ← prochain caractère
si (char = 'a')
alors goto q_1
sinon goto q_r

q_1 : Char ← prochain caractère
si (char = 'b' | char = 'c')
alors goto q_1
sinon si (char = 'EOF')
alors accepter
sinon goto q_r

q_r : rejeter

Remarque

Cette implémentation devient rapidement difficile à gérer manuellement mais une fois automatisé sera plus rapide que l'analyseur à base de table.

Conclusion

L'analyseur lexical prépare également le terrain pour l'analyse syntaxique. Il peut reconnaître les mots clés du langage. Pour cela, il y a deux possibilités :

1) Soit une suite explicite d'états reconnaissant chaque mot clé doit être introduite.
2) Soit, les identificateurs doivent être reconnus et comparés à une table des symboles contenant les mots clés du langage.

Partie 2 - Analyse syntaxique

1. Introduction

L'analyseur syntaxique analyse la suite de symboles issus de l'analyseur lexical et vérifie si cette suite de symboles est conforme à la syntaxe du langage définie par les règles de Backus-Naur. Pour cela, l'analyseur syntaxique essaye de construire l'arbre syntaxique correspondant au code. Dans cet arbre, les feuilles correspondent aux symboles issus de l'analyse lexicale et les nœuds intermédiaires correspondent aux objets grammaticaux.

Remarque

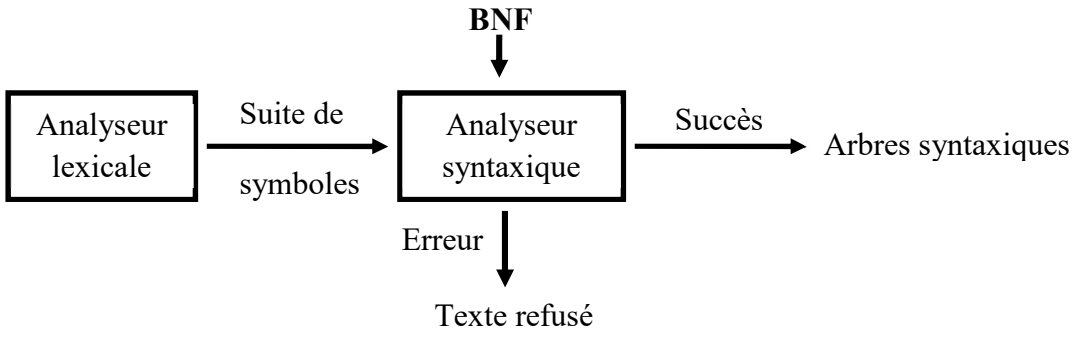
Si l'analyseur syntaxique ne parvient pas à construire l'arbre syntaxique du programme compilé alors la syntaxe du programme est erronée.

Définition

Soient $G = (V_T, V_N, S, P)$ et $x \in V_T^*$. L'analyse syntaxique est le processus d'établissement d'une correspondance entre l'axiome S et une chaîne terminale x :

1) $x \in L(G)$ ➔ construction de l'arbre syntaxique associé à la chaîne terminale.

2) $x \notin L(G)$ ➔ chaîne refusée.



Analyseur syntaxique

Conclusion

La construction des arbres syntaxiques fait apparaître la nécessité d'avoir défini la grammaire du langage sans ambigüité afin qu'à aucun moment, une suite de symboles issue de l'analyse lexicale puisse correspondre à plusieurs règles de production différentes, ce qui conduirait à une interprétation du sens différente à celle voulue par le programmeur.

Exemple de grammaire ambigüe

Soit $G = (V_T, V_N, S, P)$, V_T = {if, then, else} , V_N = {Bool, Instr}, S = Instr ,

P : Instr ➔ if bool then instr else instr

Instr ➔ if bool then instr

En se basant sur cette grammaire, la phrase : «**if bool then if bool then instr else instr**» est-elle ambiguë?

La phrase admet deux arbres syntaxiques différents. Le concepteur du langage veut que l'instruction soit interprétée comme le 2ème cas. D'où la nécessité de supprimer l'ambigüité en changeant les productions.

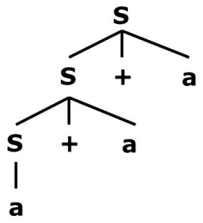
Définition

- Analyseur syntaxique non déterministe : aucun critère n'est imposé pour choisir la règle de production à appliquer.
- Analyseur syntaxique déterministe : des critères sont imposés pour choisir sans ambigüité la règle de production à appliquer.
- Analyseur syntaxique unique : pour une chaîne terminale donnée, un seul arbre syntaxique est fourni.
- Analyseur syntaxique multiple : plusieurs arbres syntaxiques possibles pour une même chaîne terminale donnée.
- Analyse syntaxique descendante : L'analyseur cherche, à partir de l'axiome et à l'aide des règles de la grammaire, à former la phrase donnée.
- Analyse syntaxique ascendante : L'analyseur cherche, à partir de la phrase et à l'aide des règles de la grammaire, à remonter à l'axiome.

Exemple

G = ({a, b, '+'}, {S}, S, R) , R: S $\rightarrow$ S + a | S + b | a | b, Soit x = a + b + b

L'analyseur syntaxique essayera de construire l'arbre de dérivation correspondant. L'analyse syntaxique descendante construit par exemple l'arbre suivant :



Les règles sont appliquées au hasard mais le résultat est erroné. La dernière règle est remplacée par une autre. Si le résultat est toujours faux, la règle précédente est analysée. Le concepteur devra donc choisir des analyseurs syntaxiques déterministes, uniques et descendants. D'où le choix des grammaires LL(K). En effet, il existe une classe de grammaire adaptée à l'analyse descendante déterministe : la classe des grammaires LL(k). Le premier L (Left) signifie que l'analyse des phrases se fait de la gauche vers la droite. Le deuxième L indique qu'il faut effectuer des dérivations à gauche avec une inspection de k symboles terminaux en avance sur le symbole courant traité.

La réalisation d'analyseurs syntaxiques pour des grammaires LL(k) avec k>1 est difficile et ces grammaires ne sont pas performantes. Les grammaires de type LL(0) sont trop restrictives. Par conséquent le type LL(1) est le plus utilisé. Le but de cette classe de grammaires est de toujours connaître la règle de production à appliquer et ceci en se basant uniquement sur le terminal suivant non encore traité de la phrase en cours d'examen.

3. **Calcul de PREMIER**

Pour toute chaîne α composée de symboles terminaux et non-terminaux, on cherche PREMIER(α) :

PREMIER(α) est l'ensemble de tous les terminaux qui peuvent commencer une chaîne qui se dérive de α. On cherche toutes les lettres a telle qu'il existe une dérivation $\alpha =^* \rightarrow a\beta$ ($\beta \in V^*$)

PREMIER(α) = $\{a \in V_T / \exists \beta \in V^*, \alpha^* \rightarrow a\beta\} \cup \{\varepsilon \text{ si } \alpha^* \rightarrow \varepsilon\}$

Exemple

$$\begin{cases} S \rightarrow Ba \\ B \rightarrow cP \mid bP \mid P \mid \varepsilon \\ P \rightarrow dS \end{cases}$$

$S \ ^* \rightarrow a$ alors $a \in$ PREMIER(S).

$S \ ^* \rightarrow cPa$ alors $c \in$ PREMIER(S).

$S \ ^* \rightarrow bPa$ alors $b \in$ PREMIER(S).

$S \ ^* \rightarrow dSa$ alors $d \in$ PREMIER(S).

S n'engendre pas le mot vide d'où PREMIER(S) = {a, b, c, d}

2.1. Calcul de Premier pour un symbole

1. Si X est un non terminal et $X \rightarrow Y_1Y_2\ldots Y_n$ est une production de la grammaire (avec Y_i symbole terminal ou non-terminal) alors :

i) ajouter les éléments de PREMIER(Y_1) sauf ε dans PREMIER(X).

ii) S'il existe j ($j \in \{2, \ldots, n\}$) tel que pour tout i = 1, …, j-1 on a $\varepsilon \in$ PREMIER(Y_i), alors ajouter les éléments de PREMIER(Y_j) sauf ε dans PREMIER(X).

iii) Si pour tout i = 1, …, n, $\varepsilon \in$ PREMIER(Y_i), alors ajouter ε dans PREMIER(X).

2. Si X est un non terminal et $X \rightarrow \varepsilon$ est une production alors ajouter ε dans PREMIER(X).

3. Si X est un terminal alors PREMIER(X) = {X}.

4. Recommencer jusqu'à ce qu'on n'ajoute rien de nouveau dans les ensembles PREMIER.

Calcul de Premier pour une chaîne

1. Pour calculer Premier pour une chaîne $X_1X_2\ldots X_n$:

i) Ajouter à Premier($X_1X_2\ldots X_n$) tous les symboles de Premier(X_1) différents de ε.

ii) Si ε est dans Premier(X1), ajouter également les symboles de Premier(X2) différents de ε, ...

iii) Finalement, si quelque soit i, Premier(X_i) contient ε, ajouter ε à Premier($X_1X_2\ldots X_n$).

3. Calcul de SUIVANT

Pour tout non-terminal A, SUIVANT(A) est l'ensemble de tous les symboles terminaux a qui peuvent apparaître immédiatement à droite de A dans une dérivation : $S \xrightarrow{*} \alpha A a \beta$ où α et β sont des chaînes de symboles grammaticaux.

Suivant(A) = $\{\mathbf{a} \in V_T / \exists\ \alpha, \beta \in V^*, S \xrightarrow{*} \alpha A \mathbf{a} \beta\}$

Exemple

G = ({a, b, c, d, ε}, {S, B, P}, S, R)

Où R : $\begin{cases} S \rightarrow Sc \mid Ba \\ B \rightarrow Pa \mid bPb \mid P \mid \varepsilon \\ P \rightarrow dS \end{cases}$

$S \rightarrow S\ c$ alors c ∈ SUIVANT(S)

$S \rightarrow Ba \rightarrow Pa \rightarrow dSa$ alors a ∈ SUIVANT(S)

$S \rightarrow Ba \rightarrow bPba \rightarrow bdSba$ alors b ∈ SUIVANT(S)

3. 1. Algorithme de construction des ensembles SUIVANT

Pour calculer SUIVANT(A) pour tous les non-terminaux, appliquer les règles suivantes jusqu'à aucun terminal ne puisse être ajouté aux ensembles SUIVANT :

1. Mettre $ dans SUIVANT(S) où S est l'axiome de départ de la grammaire et $ est le marqueur indiquant la fin du texte source.

2. S'il y a une production $A \rightarrow \alpha B \beta$ où B est un non-terminal, alors le contenu de PREMIER(β) sauf ε est ajouté à SUIVANT(B).

SUIVANT(B) ⊇ PREMIER(β) \ ε en effet : Si a ∈ Premier(β)\{ε} ($\beta \xrightarrow{*} aw$)

$S \xrightarrow{*} \alpha' A \beta' \rightarrow \alpha' \alpha B \beta \beta' \xrightarrow{*} \alpha' \alpha B a w \beta'$

3. S'il existe une production $A \rightarrow \alpha B$ ou $A \rightarrow \alpha B \beta$ telle que ε ∈ PREMIER(β) (ie $\beta \xrightarrow{*} \varepsilon$) alors les éléments de SUIVANT(A) sont ajoutés à SUIVANT(B).

SUIVANT(B) ⊇ SUIVANT(A) en effet : si a ∈ SUIVANT(A) :

$S \xrightarrow{*} \alpha' A a \beta' \rightarrow \alpha' \alpha B a \beta$ alors a ∈ SUIVANT(B).

$S \xrightarrow{*} \alpha' A a \beta' \rightarrow \alpha' \alpha B \beta a \beta' \xrightarrow{*} \alpha' \alpha B a \beta'$

car ε ∈ PREMIER(β) alors a ∈ SUIVANT(B).

4. Table d'analyse

Exemple 1

Soit la grammaire G définie par : S → aSb | aSc | d. Le mot analysé est w = *aaaaadbbcbbbc*.

Pour savoir quelle règle utilisée, il faut connaître la **dernière** lettre du mot.

Exemple 2

Reprenons la grammaire suivante : E → T E' E' → + T E' | ε T → F T' T' → * F T' | ε F → (E) | nb. Le mot analysé est w = 3*4+10*(5+11). La question posée est quelles sont les règles qui vont être utilisées?

4. 1. Construction d'analyseur

L'algorithme suivant peut être utilisé pour construire une table pour l'analyse prédictive d'une grammaire G. L'idée sous-tendant cet algorithme est le suivant.

• Soit A → α une production et a dans Premier(α).

• L'analyseur développe A en α chaque fois que le symbole d'entrée courant est a.

• Une production se produit quand α = ε ou α *→ ε. Dans ce cas, nous devons également développer A en α si le symbole d'entrée courant est dans SUIVANT(A) ou si le $ d'entrée a été atteint et que $ est dans SUIVANT(A).

4. 2. Algorithme

Donnée : une grammaire G.

Résultat : une table d'analyse M pour G.

Méthode :

• Pour chaque production A → α de la grammaire :

- Pour chaque terminal a dans Premier(α), ajouter A → α à M[A; a]

• Si ε est dans PREMIER (α) :

- Ajouter A → α à M[A; b] pour chaque terminal b dans Suivant(A)
- Si $ est dans Suivant(A), ajouter A → α à M[A; $].

• Faire de chaque entrée indéfinie de M une erreur de syntaxe.

Exemple

$E \to T\,E'$ $E' \to +\,T\,E'$ $E' \to \varepsilon$ $T \to F\,T'$ $T' \to *\,F\,T'$ $T' \to \varepsilon$ $F \to (E)$ $F \to nb$

Non-terminal	nb	+	*	(	)	$
E	$E \to TE'$			$E \to TE'$		
E′		$E' \to +TE'$			$E' \to \varepsilon$	$E' \to \varepsilon$
T	$T \to FT'$			$T \to FT'$		
T′		$T' \to \varepsilon$	$T' \to *FT'$		$T' \to \varepsilon$	$T' \to \varepsilon$
F	$F \to nb$			$F \to (E)$		

5. Analyseur syntaxique dirigé par table

• Un **tampon d'entrée** : contient la chaîne à analyser, suivie de $, symbole utilisé comme marqueur d'extrémité droite qui indique la fin de la chaîne d'entrée.

• Une **pile** : contient une séquence de symboles grammaticaux, avec $ marquant le fond de pile. Initialement, la pile contient l'axiome de la grammaire au dessus du $.

• Une **table d'analyse** : tableau à deux dimensions M[A; a] où A est un non-terminal et a un terminal ou le symbole $.

• Un **flot de sortie** : ce qui est récupéré.

5. 1. Principe

L'analyse syntaxique est contrôlée par un programme qui a le comportement suivant, sachant que X est le symbole au sommet de la pile, et a le symbole d'entrée courant. Ces deux symboles déterminent l'action de l'analyseur. Il y a trois possibilités :

• Si X = a = $: l'analyseur s'arrête et annonce la réussite finale de l'analyse.

• Si X = a ≠ $: l'analyseur enlève X de la pile et avance son pointeur d'entrée sur le symbole suivant.

• Si X est un non-terminal, le programme consulte l'entrée M[X; a] de la table d'analyse M. Cette entrée sera soit une X-production de la grammaire soit une erreur.

> • Si par exemple, M[X; a] = {X → UVW}, l'analyseur remplace X en sommet de pile par WVU (avec U au sommet). Nous supposerons que l'analyseur se contente pour tout résultat d'imprimer la production utilisée.
>
> • Si M[X; a] = erreur, l'analyseur appelle une procédure de récupération sur l'erreur.

5. 2. Algorithme d'analyse

Donnée : une chaîne w et une table d'analyse M pour une grammaire G.

Résultat : si w est dans L(G), une dérivation gauche pour w, sinon une indication d'erreur.

Méthode : initialement, l'analyseur est dans une configuration dans laquelle il y a $S dans la pile, avec S l'axiome de G au sommet, et w$ dans son tampon d'entrée.

Algorithme

• Positionner le pointeur source ps sur le premier symbole de w$.

• Répéter jusqu'à ce que X = $.

- Soit X le symbole en sommet de pile et a le symbole pointé par ps.
- Si X est un terminal ou $ alors :
  - Si X = a alors enlever X de la pile et avancer ps.
  - Sinon Erreur ()
- Sinon (X est un non-terminal) :

  Si M[X; a] = $X \rightarrow Y_1Y_2 \dots Y_n$ alors

  Debut :

  Enlever X de la pile

  Mettre $Y_n; Y_{n-1}; \dots ; Y_1$ sur la pile (avec Y_1 au sommet)

  Emettre en sortie la production $X \rightarrow Y_1Y_2 \dots Y$

  Fin

  Sinon Erreur ()

Exemple

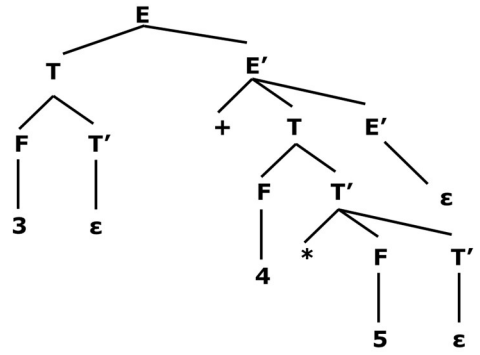
$E \rightarrow T\ E'$ $E' \rightarrow +\ T\ E'$ $E' \rightarrow \varepsilon$ $T \rightarrow F\ T'$ $T' \rightarrow *\ F\ T'$ $T' \rightarrow \varepsilon$ $F \rightarrow (E)$ $F \rightarrow nb$

Le mot w = 3 + 4 * 5

Pile	Entrée	Sortie
$ E	3 + 4 * 5 $	$E \rightarrow T\ E'$
$ E' T	3 + 4 * 5 $	$T \rightarrow F\ T'$

$ E' T' F	3 + 4 * 5 $	F → nb
$ E' T' **3**	**3** + 4 * 5 $	
$ E' T'	+ 4 * 5 $	T' → ε
$ E'	+ 4 * 5 $	E' → + T E'
$ E' T +	+ 4 * 5 $	
$ E' T	4 * 5 $	T → F T'
$ E' T' F	4 * 5 $	F → nb
$ E' T' **4**	**4** * 5 $	
$ E' T'	* 5 $	T' → * F T'
$ E' T' F *	* 5 $	
$ E' T' F	5 $	F → nb
$ E' T' **5**	**5** $	
$ E' T'	$	T' → ε
$ E'	$	E' → ε
$	**$**	Analyse réussie

Par conséquent, l'analyse syntaxique s'est bien déroulée er l'arbre de dérivation est construit et est définit ci-dessous :



Arbre syntaxique pour 3 + 4 *5.

Remarque

L'algorithme précédent peut il être appliqué à toutes les grammaires?

Si la table d'analyse comporte des entrées multiples (plusieurs productions pour une même case M[A, α]), le choix de la production adéquate sera difficile.

Définition

Une grammaire est dite de type LL(1) si la table d'analyse associée n'a aucune case définie de façon multiple.

Remarque

La grammaire n'est pas de type LL(1). En effet, pour pouvoir choisir entre la production A $\rightarrow$ cd et la production A $\rightarrow$ c, il faut lire la lettre qui suit celle que l'on pointe (donc deux symboles de prévision sont nécessaires).

Théorème

Une grammaire ambiguë ou récursive à gauche ou non factorisée à gauche n'est pas LL(1).

6. Récursivité à gauche

Une grammaire est immédiatement récursive à gauche si elle contient un non-terminal A tel qu'il existe une production A $\rightarrow$ Aα où α est une chaîne quelconque.

Exemple

S $\rightarrow$ ScA | B A $\rightarrow$ Aa | ε B $\rightarrow$ Bb | d | e

Cette grammaire contient plusieurs récursivités à gauche immédiates (ou directes).

6. 1. Elimination de la récursivité à gauche immédiate

Remplacer toute règle de la forme A $\rightarrow$ Aα | β avec $\alpha \in (V_T \cup V_N)^+$ et $\beta \in (V_T \cup V_N)^*$ par les deux règles :

$A \rightarrow \beta\ A'$

$A' \rightarrow \alpha\ A' \mid \varepsilon$

Algorithme

Dans un premier temps, on groupe les A-productions comme suit :

$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \ldots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \ldots \mid \beta_n$

où aucun β_i ne commence par un A. Puis on remplace les A-productions par :

$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \ldots \mid \beta_n A'$

$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \ldots \mid \alpha_m A' \mid \varepsilon$

Théorème

La grammaire ainsi obtenue reconnaît le même langage que la grammaire initiale.

7. Factorisation à gauche

Exemple

$S \rightarrow$ bacdAbd | bacdBcca

$A \rightarrow$ aD

$B \rightarrow$ cE

$C \rightarrow \ldots$

Au départ pour savoir s'il faut choisir $S \rightarrow$ bacdAbd ou $S \rightarrow$ bacdBcca, il faut lire la 5ième lettre du mot. Dès le départ, quelle production va être choisie? Ce qui est incompatible avec les grammaires LL(1).

Cas général

Si $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ sont deux A-productions et si l'entrée commence par une chaîne non vide dérivée de α, nous ne savons pas s'il faut développer A en $\alpha\beta_1$ ou A en $\alpha\beta_2$. Il est possible de différer cette décision en étendant A en $\alpha A'$. Alors après avoir lu la chaîne dérivée de α, nous étendons A' en β_1 ou en β_2.

$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ devient $A \rightarrow \alpha A'$ $\quad A' \rightarrow \beta_1 \mid \beta_2$

7. 1. Algorithme

Pour chaque non-terminal A, trouver le plus long préfixe α commun à deux de ses alternatives ou plus.

Si $\alpha \neq \varepsilon$, remplacer $A \rightarrow \alpha\beta_1 \mid \ldots \mid \alpha\,\beta_p \mid \gamma_1 \mid \ldots \mid \gamma_p$

(où les γ_i ne commencent pas par α) par les deux règles :

$A \rightarrow \alpha A' \mid \gamma_1 \mid \ldots \mid \gamma_p$

$A' \rightarrow \beta_1 \mid \ldots \mid \beta_p$

fin si

Recommencer jusqu'à ne plus en trouver.

Bibliographie

Emmanuel Desmontils (IUP-MIAGE de Nantes)
Thomas Pietrzak, Université Paul Verlaine / Metz
…